

Introduction

Every winter, air pollution in North India becomes a severe health hazard, largely due to stubble burning in neighboring regions like Punjab , Haryana & Western UP. While we have plenty of weather data available, it is usually scattered across different files and websites.

This project, **EcoPulse**, is an attempt to solve this by creating a centralized system. We are building a tool that doesn't just predict pollution levels using Machine Learning but also stores every prediction in a structured MySQL database. By using a proper DBMS instead of just CSV files, we can ensure data is safe, organized, and can be used to generate automatic alerts along with possible reason for pollution levels.

Problem Statement

- **The Problem:** Currently, analyzing the link between farm fires ,vehicle pollution ,construction dust and AQI requires manually opening multiple spreadsheets, which is slow and prone to errors.
- **The Solution:** We need a Database Management System (DBMS) to enforce constraints (like valid dates) and use backend logic to automatically flag "Severe" days without human calculation.

Objectives of the Project

The main goals of our project are:

1. To design a proper **E-R (Entity-Relationship) Model** for storing environmental data.
2. To connect a **Python** interface with a **MySQL** backend for real-time data entry.
3. To implement **Stored Procedures** (PL/SQL) that handle business logic, like generating alerts.

Scope of the Project

- The system allows users to load historical data (training data) and make new predictions based on current weather. Basically the possible reason behind pollution.

The DBMS Architecture (The "Skeleton")

Project uses a Relational Database Management System (RDBMS) hosted on MySQL Server 8.0+.

A. The Schema (Database Design)

The database is named `PollutionDB`. It uses a **Star Schema** variation with three main tables to separate raw data from analytical data.

Table 1: `Raw_AQI_Data` (The Input Log)

- **Purpose:** Stores every single prediction attempt made by the user.
- **Columns:**
 - `RecordID` (Primary Key, INT, Auto-Increment): Unique ID for every scan.
 - `RecordDate` (DATETIME): Timestamp of when you ran the code.
 - `FireCount` (INT): Number of farm fires (Input).
 - `WindSpeed` (FLOAT): Speed in km/h (Input).
 - `WindDir` (INT): Direction in degrees (Input).
 - `Temperature` (FLOAT): In Celsius (Input).
 - `PredictedAQI` (FLOAT): The output from your AI.

Table 2: `Model_Training_Log` (The AI Memory)

- **Purpose:** Keeps track of how well the AI is performing (Accuracy).
- **Columns:**
 - `LogID` (Primary Key): Unique ID.
 - `TrainingDate`: When the model was trained.
 - `RMSE` (Float): Root Mean Squared Error (The "Math" score of how wrong the model is on average).
 - `ModelVersion`: To track updates.

Table 3: **Daily_Summary** (The Report Card)

- **Purpose:** Automatically calculated summary (Average AQI per day).
- **Columns:**
 - **SummaryDate** (Primary Key): The specific date.
 - **AvgAQI**: Average pollution for that day.
 - **MaxFireCount**: Worst fire count recorded that day.

B. The Automation (Stored Procedures)

We have a "Robot" inside the database called **Update_Daily_Summary**.

- **Trigger:** Whenever you save data, this procedure wakes up.
- **Logic:** It calculates the average AQI for the current date from **Raw_AQI_Data** and updates the **Daily_Summary** table automatically.

The Machine Learning Mathematics (The "Brain")

Our AI model is a **Gradient Boosting Regressor (GBR)**. It is not just "guessing"; it is using Calculus and Statistics.

The Algorithm: Ensemble Learning

Instead of one "smart" brain, your model uses 100 "weak" brains (Decision Trees) that vote together.

The Math Formula: The final prediction ($F(x)$) is the sum of a baseline prediction plus the weighted corrections of many small trees.

$$F(x) = \text{Initial Guess} + \sum_{i=1}^{100} (\eta \times h_i(x))$$

- $F(x)$: The Predicted AQI
- Initial Guess: The average AQI of all historical data.
- η (Eta): The "Learning Rate" (how fast it learns, set to 0.1).
- $h_i(x)$: The output of the i -th Decision Tree.

How it "Learns" (The Gradient Descent)

The model tries to minimize a **Loss Function** (Squared Error):

$$L(y, F(x)) = \frac{1}{2}(y - F(x))^2$$

- y : The *Real* AQI (from history).
- $F(x)$: The *Predicted* AQI.

Every time it trains, it calculates the **Residual** (Difference between Truth and Prediction):

$$\text{Residual} = y - F(x)$$

The next tree is mathematically built to predict this *Residual*, effectively "fixing" the mistake of the previous tree.

How They Are Linked (The "Nervous System")

Our **Python Script** and **MySQL Database** are two separate worlds. They talk through a "Bridge" called a **Connector**.

The Flow:

1. **Input**
2. **Calculation**
3. **Connection:** Python uses the `mysql.connector` library to "dial" the database.
 - **Host:** `localhost` (Your Mac).
 - **User:** `root`.
 - **Password:** `password`.
 - **Port:** `3306` (The specific "door" MySQL listens on).
4. **Transaction:**
 - Python constructs a SQL sentence: `INSERT INTO Raw_AQI_Data VALUES ( . . . )`.
 - It sends this packet to Port 3306.
5. **Commit:** MySQL receives it, writes it to the hard drive, and sends back a "Success" signal.

Where is the Data Actually Stored?

Physically, your database is not "in the cloud." It is a folder on your Mac's hard drive.

- **Location:** `/usr/local/mysql/data/`
- **File Type:** Inside that folder, there is a subfolder named `PollutionDB`. Inside that, your tables (`Raw_AQI_Data`, etc.) are stored as **.ibd files** (InnoDB Data files).